



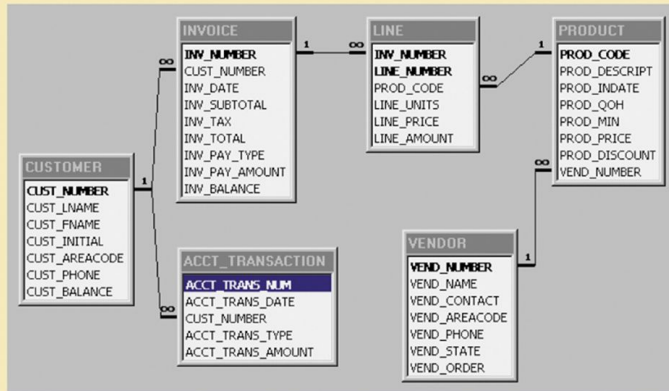
Module 12: Transaction Management

CPE 011 Database Management System

Slides based on “Database Systems: Design, Implementation, and Management 8th Edition” Chapter 10: Concurrency Control Slides by Rob, P. and Coronel, C.

What is a Transaction?

FIGURE 10.1 The Ch10_SaleCo database relational diagram



- Any action that reads from and/or writes to a database may consist of:
 - Simple SELECT statement to generate list of table contents
 - Series of related UPDATE statements to change values of attributes in various tables
 - Series of INSERT statements to add rows to one or more tables
 - Combination of SELECT, UPDATE, and INSERT statements

Defining a Transaction

- Transaction is logical unit of work that must be either entirely completed or aborted
- Successful transaction changes database from one consistent state to another
 - One in which all data integrity constraints are satisfied
- Most real-world database transactions are formed by two or more database requests
 - Equivalent of a single SQL statement in an application program or transaction

Evaluating Transaction Results

- **Not all transactions update database**
- SQL code represents a transaction because database was accessed
- Improper or incomplete transactions can have devastating effect on database integrity
 - Some DBMSs provide means by which user can define enforceable constraints
 - Other integrity rules are enforced automatically by the DBMS

Transaction Properties

- Atomicity
 - Requires that all operations (SQL requests) of a transaction be completed
- Consistency
 - Indicates the permanence of database's consistent state
- Isolation
 - Data used during execution of a transaction cannot be used by second transaction until first one is completed
- Durability
 - Indicates permanence of database's consistent state
- Serializability
 - Ensures that concurrent execution of several transactions yields consistent results

Transaction Management with SQL

- ANSI (American National Standards Institute) has defined standards that govern SQL database transactions
- Transaction support is provided by two SQL statements: COMMIT and ROLLBACK
- **ANSI standards require that, when a transaction sequence is initiated by a user or an application program, it must continue through all succeeding SQL statements until one of four events occurs:**
 - COMMIT statement is reached
 - ROLLBACK statement is reached
 - End of program is reached
 - Program is abnormally terminated

Transaction Log

It stores:

- A record for the beginning of transaction.
- For each transaction component (SQL statement):
 - Type of operation being performed (update, delete, insert)
 - Names of objects affected by transaction
 - “Before” and “after” values for updated fields
 - Pointers to previous and next transaction log entries for the same transaction
- Ending (COMMIT) of the transaction

TABLE
10.1

A Transaction Log

TRL_ID	TRX_NUM	PREV_PTR	NEXT_PTR	OPERATION	TABLE	ROW_ID	ATTRIBUTE	BEFORE_VALUE	AFTER_VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	1558-QW1	PROD_QOH	25	23
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	525.75	615.73
365	101	363	Null	COMMIT	**** End of Transaction				

↑

TRL_ID = Transaction log record ID
TRX_NUM = Transaction number
(Note: The transaction number is automatically assigned by the DBMS.)

PTR = Pointer to a transaction log record ID

Concurrency Control

- Coordination of simultaneous transaction execution in a multiprocessing database system
- Objective is to ensure serializability of transactions in a multiuser database environment
- Simultaneous execution of transactions over a shared database can create several data integrity and consistency problems
 - Lost updates
 - Uncommitted data
 - Inconsistent retrievals

Lost Updates

TABLE
10.2 Normal Execution of Two Transactions

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$\text{PROD_QOH} = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH	135
5	T2	$\text{PROD_QOH} = 135 - 30$	
6	T2	Write PROD_QOH	105

TABLE
10.3 Lost Updates

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	$\text{PROD_QOH} = 35 + 100$	
4	T2	$\text{PROD_QOH} = 35 - 30$	
5	T1	Write PROD_QOH (Lost update)	135
6	T2	Write PROD_QOH	5

Uncommitted Data

TABLE
10.4 Correct Execution of Two Transactions

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$\text{PROD_QOH} = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T1	***** ROLLBACK *****	35
5	T2	Read PROD_QOH	35
6	T2	$\text{PROD_QOH} = 35 - 30$	
7	T2	Write PROD_QOH	5

TABLE
10.5 An Uncommitted Data Problem

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$\text{PROD_QOH} = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	$\text{PROD_QOH} = 135 - 30$	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

Inconsistent Retrievals

TABLE
10.6

Retrieval During Update

TRANSACTION T1		TRANSACTION T2	
SELECT	SUM(PROD_QOH)	UPDATE	PRODUCT
FROM	PRODUCT	SET	PROD_QOH = PROD_QOH + 10
		WHERE	PROD_CODE = '1546-QQ2'
		UPDATE	PRODUCT
		SET	PROD_QOH = PROD_QOH - 10
		WHERE	PROD_CODE = '1558-QW1'
		COMMIT;	

TABLE
10.7

Transaction Results: Data Entry Correction

	BEFORE	AFTER
PROD_CODE	PROD_QOH	PROD_QOH
11QER/31	8	8
13-Q2/P2	32	32
1546-QQ2	15	$(15 + 10) \rightarrow 25$
1558-QW1	23	$(23 - 10) \rightarrow 13$
2232-QTY	8	8
2232-QWE	6	6
Total	92	92

The Scheduler

- It is a **special DBMS program**
 - Purpose is to establish order of operations within which concurrent transactions are executed
- Interleaves execution of database operations to ensure serializability and isolation of transactions
- Bases its actions on concurrency control algorithms
- Ensures computer's central processing unit (CPU) is used efficiently
- Facilitates data isolation to ensure that two transactions do not update same data element at same time

Thank you for listening!
